

Data science Track Round 4

DEPI project

Used Car Price Prediction

Team Members

Name
Islam Mohamed Fathy
Mariam Wassim William
Roaa Mostafa Mohamed
Youssef Shamandy
Yehia Shady
Mohamed Gamal

Car Price Prediction Using Machine Learning

1. Introduction

1.1 Project Overview

This project focuses on developing a comprehensive machine learning system for predicting car prices based on various vehicle attributes. The automotive market requires accurate pricing mechanisms to assist buyers, sellers, and dealers in making informed decisions. This project leverages multiple regression algorithms to build a robust prediction model capable of estimating vehicle prices based on specifications and characteristics.

1.2 Objectives

The primary objectives of this project are:

- Analyze and preprocess an automotive dataset for machine learning applications
 - Identify key features that significantly influence car pricing
 - Develop and compare multiple regression models for price prediction
 - Select the optimal model based on comprehensive performance metrics
 - Create a deployable prediction system with saved models and preprocessing pipelines
-

1.3 Dataset Description

The dataset contains comprehensive information about various vehicles, including their specifications, features, and corresponding market prices.

- **Numerical Features:**
Year, mileage, engine specifications, etc.
- **Categorical Features:**
Brand, model, fuel type, transmission type, etc.

This diverse feature set provides a solid foundation for building an accurate and generalizable car price prediction model.

2. Methodology

2.1 Data Preprocessing Pipeline

2.1.1 Data Cleaning

The data cleaning process involved the following steps:

- **Column Standardization**
Converted all column names to lowercase and replaced spaces with underscores for consistency.
 - **Missing Values Handling**
Columns with excessive missing values (`complectation` , `super_gen_name`) were removed. Remaining rows with null values were dropped.
 - **Duplicate Removal**
Duplicate records were identified and removed to avoid bias in training.
 - **Feature Selection**
Irrelevant columns that did not contribute to price prediction were eliminated.
-

2.1.2 Exploratory Data Analysis (EDA)

EDA was conducted to understand data structure and feature behavior:

- **Distribution Analysis**
Histograms were plotted for continuous variables to detect skewness and anomalies.
 - **Categorical Analysis**
Count plots were used to visualize category frequency distributions.
 - **Correlation Analysis**
A correlation matrix heatmap was generated to detect relationships and multicollinearity.
 - **Skewness Assessment**
Skewness metrics were computed for all continuous features.
-

2.1.3 Outlier Detection and Treatment

A two-stage outlier handling approach was implemented:

1. **IQR Method**
Values outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ were flagged as outliers.
2. **Winsorization**
Extreme values were capped at the 1st and 99th percentiles.

3. Outlier Removal

Remaining extreme values were removed using IQR.

4. Visualization

Boxplots were used to validate outlier treatment effectiveness.

2.1.4 Feature Engineering and Statistical Testing

- **Chi-Square Test**

Applied to categorical features to test statistical significance with price ($p < 0.05$).

- **ANOVA Test**

Used to evaluate price variance across categorical groups.

- **Label Encoding**

All categorical variables were encoded using `LabelEncoder`.

- **Feature Scaling**

`MinMaxScaler` normalized numerical features to the $[0, 1]$ range.

2.2 Model Development

2.2.1 Train-Test Split

- Training Set: **80%**
 - Testing Set: **20%**
 - Random State: **42**
 - Stratified sampling was used where applicable to maintain data distribution.
-

2.2.2 Model Selection and Training

Six regression models were implemented and optimized:

1 Linear Regression

Baseline model assuming linear relationships.

- `fit_intercept`
 - `positive`
-

2 Ridge Regression (L2 Regularization)

Handles multicollinearity via coefficient shrinkage.

- `alpha` $\in [10^{-3}, 10^3]$
-

3 Lasso Regression (L1 Regularization)

Performs feature selection by zeroing weak coefficients.

- `alpha` $\in [10^{-4}, 10^1]$
 - `max_iter` = 10000
-

4 Elastic Net Regression

Combination of L1 and L2 regularization.

- `alpha`
 - `l1_ratio` $\in [0.1, 0.9]$
-

5 Decision Tree Regressor

Captures non-linear patterns and feature interactions.

- `max_depth`
 - `min_samples_split`
 - `min_samples_leaf`
 - `max_features`
-

6 Gradient Boosting Regressor

Sequential ensemble model correcting prior errors.

Configuration:

- `n_estimators` = 200
 - `learning_rate` = 0.1
 - `max_depth` = 3
 - `subsample` = 0.8
-

2.2.3 Hyperparameter Optimization

- Technique: `RandomizedSearchCV`
 - Cross-validation: **3-fold**
 - Optimization metric: **R² Score**
 - Prevents overfitting and reduces computation cost
-

2.3 Model Evaluation Metrics

Metric	Description
R² Score	Proportion of variance explained

Metric	Description
RMSE	Penalizes large prediction errors
MAE	Robust average absolute error

2.4 Model Persistence

Saved using `joblib` :

- All six trained models (`.pkl`)
- Best model (`best_model.pkl`)
- Label encoders (`label_encoders.pkl`)
- MinMaxScaler (`minmax_scaler.pkl`)
- Feature column order (`columns.pkl`)

This ensures full pipeline reproducibility.

3. Results and Discussion

3.1 Data Characteristics

- Improved feature distributions after winsorization
 - Significant categorical impact (brand, model, fuel, transmission)
 - High-quality dataset with preserved volume
-

3.2 Model Performance Comparison

Expected Performance Ranking:

1. Gradient Boosting Regressor (Best)
2. Decision Tree Regressor
3. Elastic Net / Ridge
4. Lasso Regression
5. Linear Regression (Baseline)

A bar chart visualization compares R^2 scores across models.

3.3 Key Findings

Feature Importance

- Statistically significant categorical features identified
- Strong correlations among key numerical features

Model Insights

- Regularization improved linear model stability
- Tree-based models captured non-linearity effectively
- Gradient Boosting achieved best generalization

Preprocessing Impact

- Winsorization improved stability
 - Scaling prevented feature dominance
 - Label encoding handled high-cardinality features efficiently
-

3.4 Model Deployment Considerations

- Real-time prediction ready
 - Consistent preprocessing ensured
 - Compatible with Flask / FastAPI
 - Versioned and updatable
-

3.5 Limitations and Future Work

Limitations

- No temporal or economic factors
- Limited geographic representation
- Feature availability constraints

Future Enhancements

- XGBoost / LightGBM
 - Neural networks
 - Ensemble stacking
 - Bayesian optimization
 - Monitoring & automated retraining
-

3.6 Conclusion

This project successfully implemented a **complete end-to-end machine learning pipeline** for car price prediction.

Key Achievements

- Robust preprocessing and feature validation
- Multi-model comparison
- Deployment-ready pipeline

Practical Value

The solution can be directly integrated into real-world pricing systems.

Final Remarks

The Gradient Boosting model demonstrated the best balance between accuracy and generalization, making it the optimal choice for automotive price prediction.

4. References

- Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR, 2011
- Hastie et al., *The Elements of Statistical Learning*, Springer, 2009
- Chen & Guestrin, *XGBoost*, KDD, 2016
- McKinney, *Data Structures for Statistical Computing in Python*, 2010
- Friedman, *Gradient Boosting Machine*, Annals of Statistics, 2001

importing

```
In [1]: # ===== basics =====
import re
import json
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings("ignore")

# ===== saving =====
import joblib
import pickle
from datasets import load_dataset
import ast

# ===== Scikit-Learn: =====
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    mean_squared_error, r2_score, roc_curve, auc, ConfusionMatrixDisplay
)
```



```

# ===== Scikit-Learn: =====
from sklearn.preprocessing import (
    LabelEncoder, OneHotEncoder, StandardScaler, MinMaxScaler, PolynomialFeatures
)
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin

# ===== Scikit-Learn: =====
from sklearn.linear_model import (
    LinearRegression, Lasso, Ridge, ElasticNet, LassoCV, HuberRegressor, LogisticRe
)
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.ensemble import (
    RandomForestClassifier, RandomForestRegressor, GradientBoostingClassifier
)
from sklearn.neural_network import MLPClassifier, MLPRegressor
from sklearn.svm import SVC, LinearSVC, SVR
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB, ComplementNB
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor, NearestNei

# ===== Scikit-Learn: =====
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity, linear_kernel
from xgboost import XGBClassifier
from sklearn.feature_selection import chi2

```

```

In [2]: from google.colab import drive                                # the library that connects notebook w
drive.mount('/content/drive')                                         # connection with my drive
file_path = '/content/drive/My Drive/datasets/dataset.csv' # dataset path
df = pd.read_csv(file_path) # read the dataset and convert it from CSV to dataframe
pd.set_option('display.max_columns', len(df.columns)) # adjusting style
df.rename(columns=lambda x: x.strip().lower().replace(' ', '_'), inplace=True)
df

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Out[2]:

	owners	year	price	region	mileage	mark	model	complectat
0	1	2021	1460000	Приморский край	56000	Nissan	Note	↑
1	1	2011	1460000	Орловская область	119000	Skoda	Yeti	Elega
2	2	2019	1460000	Калужская область	73100	Kia	Rio	↑
3	2	2018	1459000	Москва и Московская область	103500	Volkswagen	Polo	D
4	1	2022	1457000	Белгородская область	33641	CHERY	Tiggo 4 Pro	Ac
...	...	...	...	...	...	...	...	
432222	4	2004	287000	Республика Башкортостан	289567	Renault	Megane	↑
432223	3	1994	289777	Краснодарский край	236987	Volkswagen	Passat	↑
432224	2	2010	290000	Москва и Московская область	103630	3A3	Chance	
432225	3	2005	290000	Москва и Московская область	216856	Renault	Logan	↑
432226	3	1982	290000	Ростовская область	72000	Lada (BA3)	2106	↑

432227 rows × 21 columns



exporatory of data

In [3]: `df.info()` # get the schema, data type and memory usage

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 432227 entries, 0 to 432226
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   owners                 432227 non-null  int64
1   year                  432227 non-null  int64
2   price                 432227 non-null  int64
3   region                432227 non-null  object
4   mileage               432227 non-null  int64
5   mark                  432227 non-null  object
6   model                 432227 non-null  object
7   complectation         171896 non-null  object
8   steering_wheel        432227 non-null  object
9   gear_type             432227 non-null  object
10  engine                432227 non-null  object
11  transmission           432227 non-null  object
12  power                 432227 non-null  int64
13  displacement          429470 non-null  float64
14  characteristics       432227 non-null  object
15  color                 432227 non-null  object
16  body_type_type        432227 non-null  object
17  body_type_name        432227 non-null  object
18  super_gen_name        388112 non-null  object
19  super_gen_year_from   432227 non-null  int64
20  super_gen_year_to     421987 non-null  float64
dtypes: float64(2), int64(6), object(13)
memory usage: 69.3+ MB
```

```
In [4]: df.describe(include="all") # analyze the data
```

Out[4]:

	owners	year	price	region	mileage	mark
count	432227.000000	432227.000000	4.322270e+05	432227	432227.000000	432227
unique	NaN	NaN	NaN	87	NaN	224
top	NaN	NaN	NaN	Москва и Московская область	NaN	Lada (BA3)
freq	NaN	NaN	NaN	86985	NaN	61363
mean	2.460154	2012.028888	1.619113e+06	NaN	162046.826977	NaN
std	1.188675	7.848552	1.598853e+06	NaN	102724.051894	NaN
min	1.000000	1923.000000	2.000000e+04	NaN	1.000000	NaN
25%	1.000000	2008.000000	5.650000e+05	NaN	85000.000000	NaN
50%	2.000000	2013.000000	1.118800e+06	NaN	154123.000000	NaN
75%	4.000000	2018.000000	2.080000e+06	NaN	223000.000000	NaN
max	4.000000	2024.000000	1.000090e+07	NaN	1000000.000000	NaN

cleaning

```
In [5]: df.isna().sum().to_frame() # compute the Null value
```

```
Out[5]:
```

	0
owners	0
year	0
price	0
region	0
mileage	0
mark	0
model	0
complectation	260331
steering_wheel	0
gear_type	0
engine	0
transmission	0
power	0
displacement	2757
characteristics	0
color	0
body_type_type	0
body_type_name	0
super_gen_name	44115
super_gen_year_from	0
super_gen_year_to	10240

```
In [6]: df.drop(['complectation'], axis=1, inplace=True) # drop the unuseful columns
```

```
In [7]: df.drop(['super_gen_name'], axis=1, inplace=True) # drop the unuseful columns
```

```
In [8]: df.dropna(inplace=True)
```

```
In [9]: print(df.duplicated().sum()) # compute the duplicated value
```

```
In [10]: df.drop_duplicates(inplace=True) # drop the duplicated value
```

```
In [11]: df.head(5) # show the first 5 rows
```

```
Out[11]:
```

	owners	year	price	region	mileage	mark	model	steering_wheel
0	1	2021	1460000	Приморский край	56000	Nissan	Note	RIGHT
1	1	2011	1460000	Орловская область	119000	Skoda	Yeti	LEFT
2	2	2019	1460000	Калужская область	73100	Kia	Rio	LEFT
3	2	2018	1459000	Москва и Московская область	103500	Volkswagen	Polo	LEFT
4	1	2022	1457000	Белгородская область	33641	CHERY	Tiggo 4 Pro	LEFT

```
In [12]: columns = df.columns
columns
```

```
Out[12]: Index(['owners', 'year', 'price', 'region', 'mileage', 'mark', 'model',
               'steering_wheel', 'gear_type', 'engine', 'transmission', 'power',
               'displacement', 'characteristics', 'color', 'body_type_type',
               'body_type_name', 'super_gen_year_from', 'super_gen_year_to'],
              dtype='object')
```

```
In [13]: continuous_col = df.select_dtypes(include=['float64', 'int64']).columns
continuous_col = [col for col in continuous_col if col != 'price']
categorical_col = df.select_dtypes(include=['object', 'category']).columns #classi

print(categorical_col)
print(continuous_col)
```

```
Index(['region', 'mark', 'model', 'steering_wheel', 'gear_type', 'engine',
      'transmission', 'characteristics', 'color', 'body_type_type',
      'body_type_name'],
      dtype='object')
['owners', 'year', 'mileage', 'power', 'displacement', 'super_gen_year_from', 'super_gen_year_to']
```

counting categorical values

```
In [14]: for i in categorical_col:
          print(df.value_counts(i))
```

```

region
Москва и Московская область      84225
Санкт-Петербург и Ленинградская область  24436
Краснодарский край                21950
Приморский край                   20496
Свердловская область              16903
...
Витебская область                 8
Чукотский автономный округ        6
Западно-Казахстанская область     2
Гомельская область                1
Брестская область                 1
Name: count, Length: 87, dtype: int64
mark
Lada (BA3)      61356
Toyota          38459
Kia             28837
Hyundai         26391
Volkswagen      23058
...
Tofas           1
Tianma          1
Yema            1
ЗИЛ             1
Канонир        1
Name: count, Length: 179, dtype: int64
model
Granta          9726
Focus           8599
Rio             7586
Solaris         7289
Priora          7178
...
1200            1
124 Sport Spider  1
13 «Чайка»      1
1307            1
1310            1
Name: count, Length: 2020, dtype: int64
steering_wheel
LEFT           384571
RIGHT          35359
Name: count, dtype: int64
gear_type
FORWARD_CONTROL  258598
ALL_WHEEL_DRIVE  129307
REAR_DRIVE       32025
Name: count, dtype: int64
engine
GASOLINE        362467
DIESEL          50089
HYBRID          7087
LPG             287
Name: count, dtype: int64
transmission
AUTOMATIC       182938

```

```

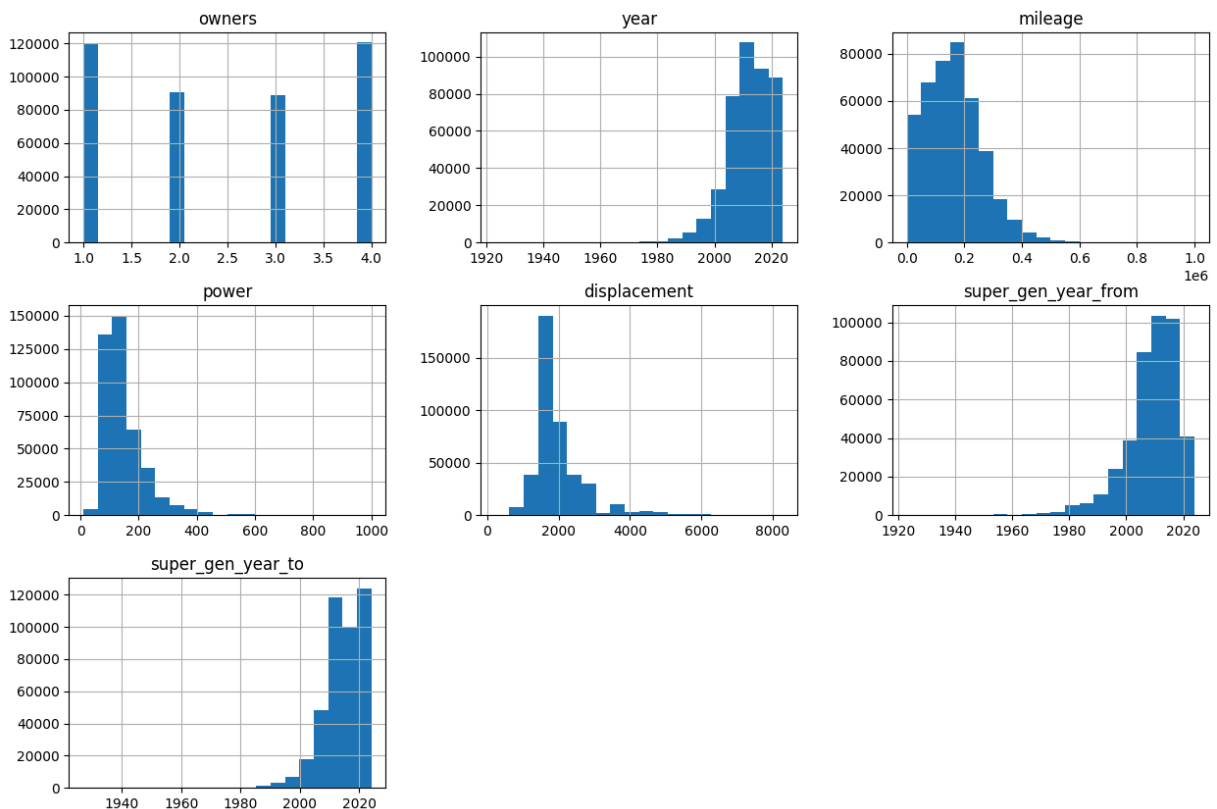
MECHANICAL      162276
VARIATOR        40806
ROBOT           33910
Name: count, dtype: int64
characteristics
1.6 MT (106 л.с.)      7444
1.6 MT (98 л.с.)      7386
1.6 MT (81 л.с.)      7320
1.6 MT (87 л.с.)      7050
2.0 AT (150 л.с.)      6712
...
6.0 AT (405 л.с.)      1
6.0 AT (420 л.с.) 4WD  1
6.0 AT (608 л.с.) 4WD  1
6.0 MT (400 л.с.)      1
6.0 AT (152 л.с.)      1
Name: count, Length: 8544, dtype: int64
color
FAFBFB      98398
040001      92934
97948F      67277
CACECB      45685
0000CC      36570
EE1D19      22301
200204      15033
007F00      13884
C49648      8858
22A0F8      5057
660099      3830
4A2197      2920
DEA522      2498
FF8649      2394
FFD600      2131
FFC0CB      160
Name: count, dtype: int64
body_type_type
SEDAN      151898
ALLROAD_5_DOORS  127277
HATCHBACK_5_DOORS  57032
WAGON_5_DOORS  23391
LIFTBACK   16076
MINIVAN    10208
COMPACTVAN  9020
HATCHBACK_3_DOORS  6907
ALLROAD_3_DOORS  5580
COUPE      3566
PICKUP_TWO  2888
VAN        2124
MICROVAN   1343
COUPE_HARDTOP  708
ROADSTER   467
CABRIO     453
ALLROAD_OPEN  274
SEDAN_HARDTOP  227
SEDAN_2_DOORS  149
PICKUP_ONE   110

```

HATCHBACK_4_DOORS	75
FASTBACK	68
PICKUP_ONE_HALF	66
TARGA	12
WAGON	7
LIMOUSINE	2
WAGON_3_DOORS	2
Name: count, dtype: int64	
body_type_name	
Седан	147877
Внедорожник 5 дв.	124896
Хэтчбек 5 дв.	54094
Универсал 5 дв.	19043
Лифтбек	15000
...	
Внедорожник 5 дв. Autech Crossover	1
Внедорожник 5 дв. SRT8	1
Внедорожник 5 дв. Signia	1
Внедорожник 5 дв. Raptor	1
Хэтчбек 3 дв. Moderno	1
Name: count, Length: 310, dtype: int64	

visualisation

```
In [15]: dist = df[continuous_col] #represent the numerical data distrubutions
dist.hist(figsize=(15, 10), bins=20) #visualize histogram using matplotlib
plt.show()
```

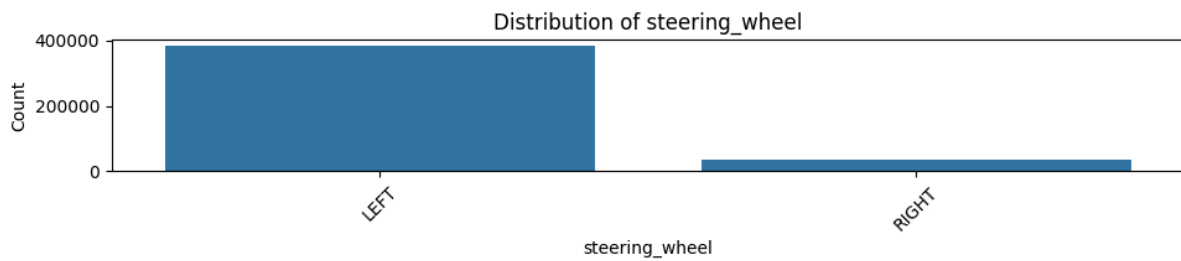
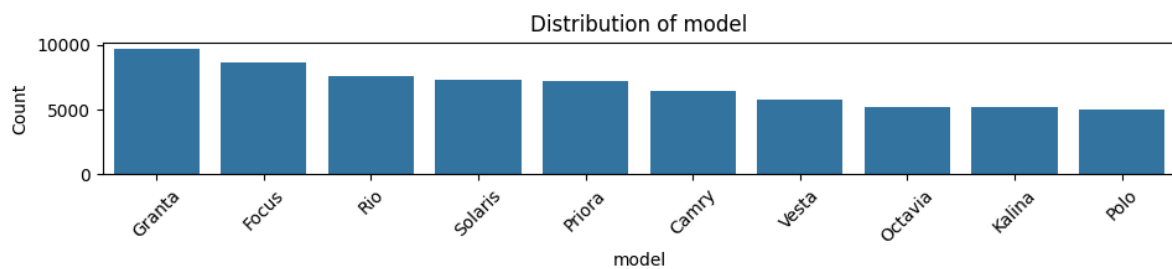
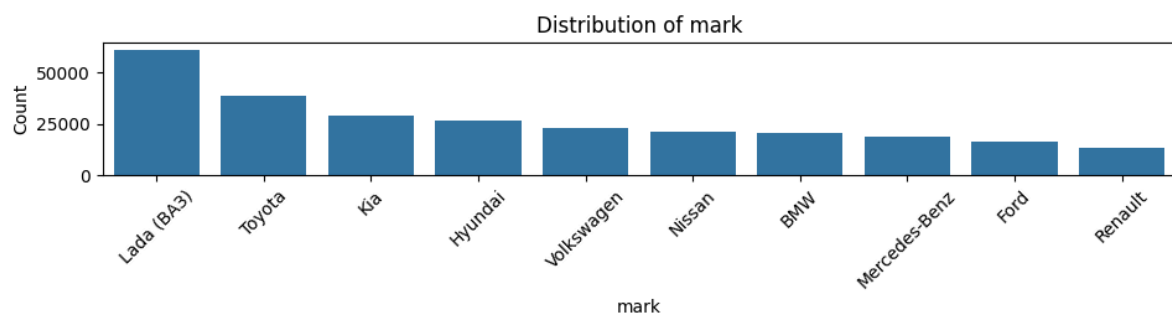
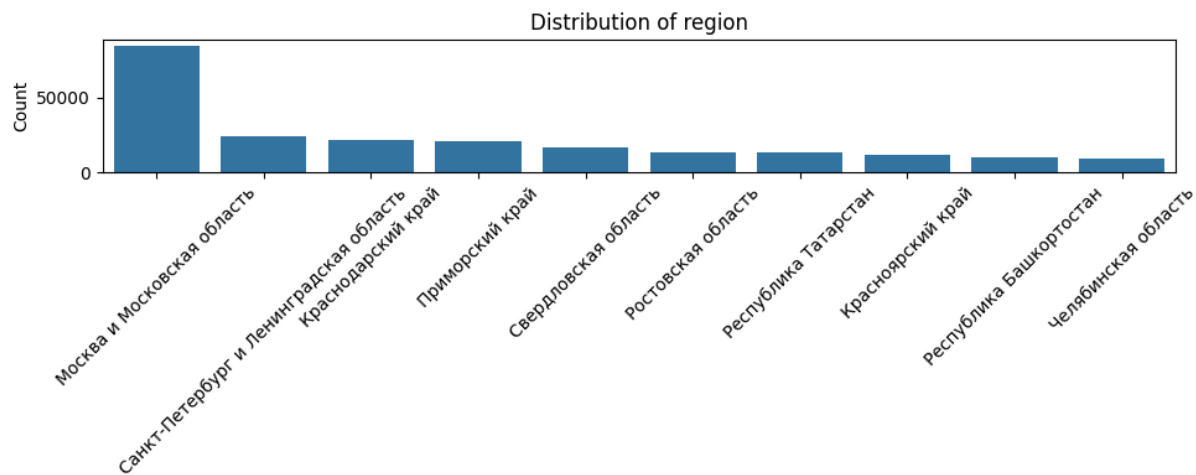


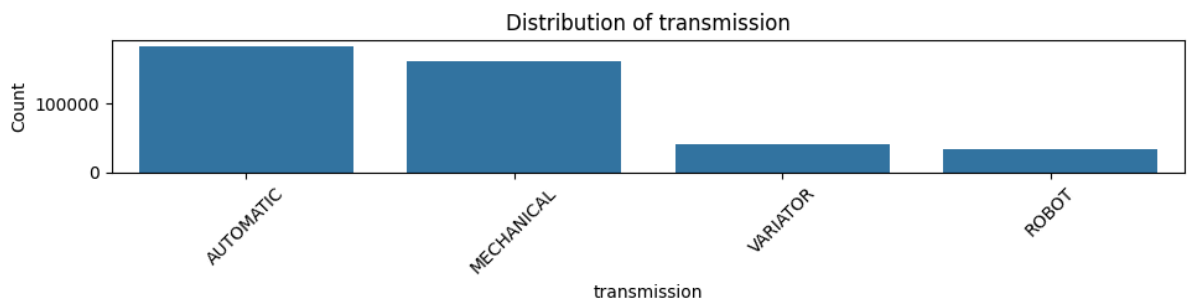
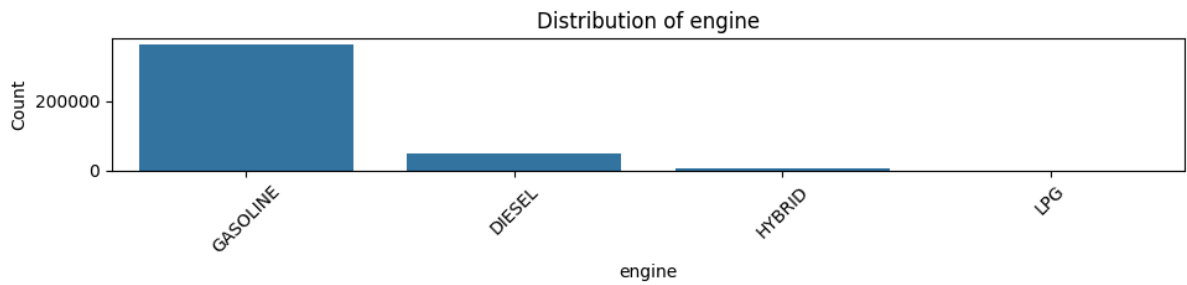
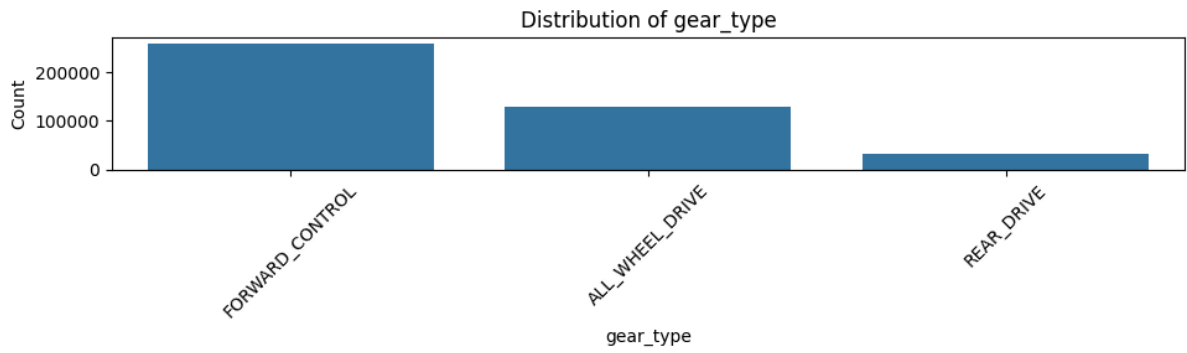
```
In [16]: fig, axes = plt.subplots(len(categorical_col), 1, figsize=(10, 4 * len(categorical_col)))
for i, col in enumerate(categorical_col):
```

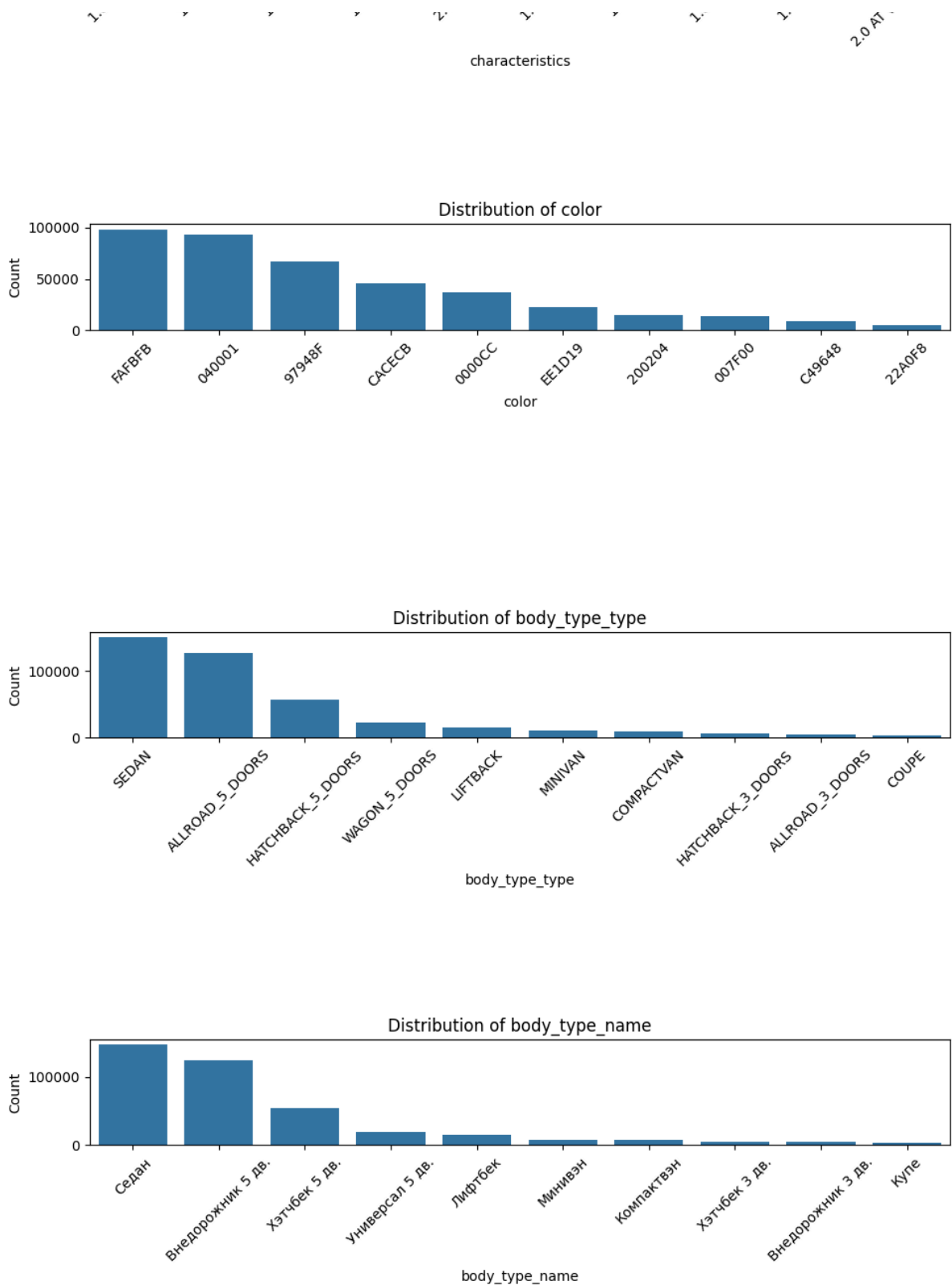


```
ax = axes[i]
sns.countplot(data=df, x=col, order=df[col].value_counts().index[:10], ax=ax)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45)
ax.set_title(f'Distribution of {col}')
ax.set_xlabel(col)
ax.set_ylabel('Count')

plt.tight_layout()
plt.show()
```



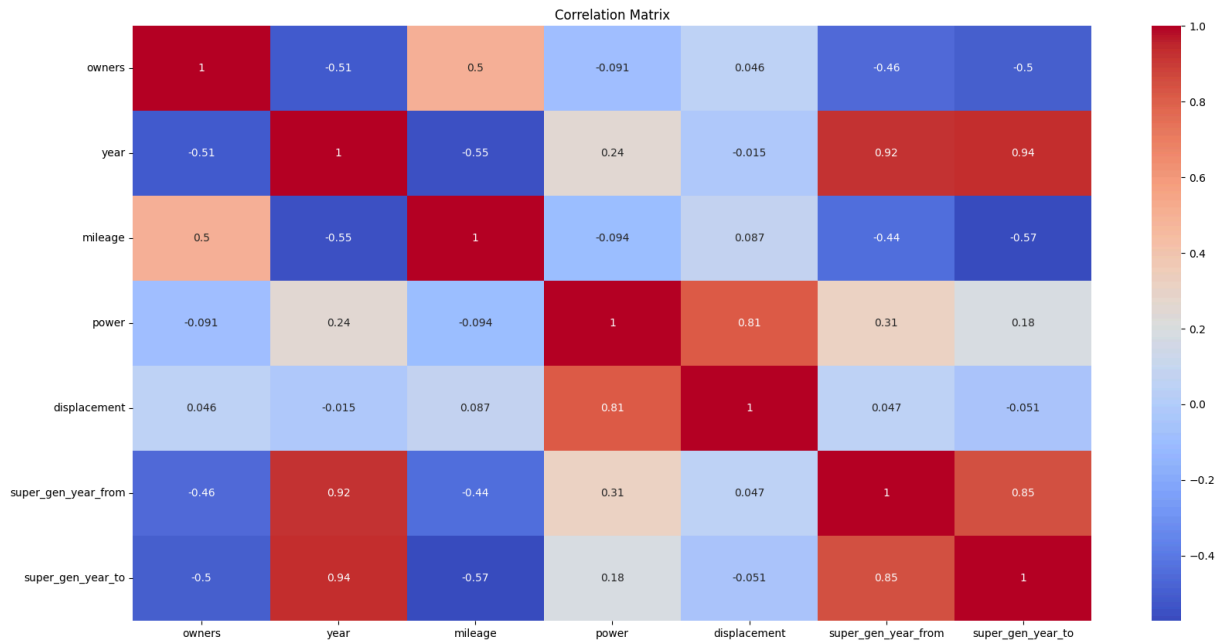




correlation matrix

```
In [17]: corr_matrix = df[continuous_col].corr()  
plt.figure(figsize=(20, 10))  
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
```

```
plt.title("Correlation Matrix")
plt.show()
```



```
In [18]: df[continuous_col].skew()
```

```
Out[18]:
```

	0
owners	0.003006
year	-1.381719
mileage	0.897651
power	1.912016
displacement	2.264154
super_gen_year_from	-1.508570
super_gen_year_to	-1.407869

dtype: float64

detecting outliers

```
In [19]: def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data[column] < lower_bound) | (data[column] > upper_bound)]

for col in continuous_col:
```

```

outliers = detect_outliers_iqr(df, col)
print(f"Number of outliers in {col}: {len(outliers)}")

```

```

Number of outliers in owners: 0
Number of outliers in year: 8532
Number of outliers in mileage: 6006
Number of outliers in power: 20429
Number of outliers in displacement: 60736
Number of outliers in super_gen_year_from: 14722
Number of outliers in super_gen_year_to: 8870

```

```

In [20]: from scipy.stats.mstats import winsorize
import pandas as pd

def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
    return outliers, lower_bound, upper_bound

def apply_winsorization(data, continuous_cols, lower_pct=0.01, upper_pct=0.01, apply=True):
    df_copy = data.copy()
    summary = {}

    for col in continuous_cols:
        outliers, low, high = detect_outliers_iqr(df_copy, col)
        n_out_before = len(outliers)

        if not apply:
            summary[col] = {
                "outliers_before": n_out_before,
                "outliers_after": n_out_before,
                "winsorized": False
            }
            continue

        df_copy[col] = winsorize(df_copy[col], limits=[lower_pct, upper_pct])
        outliers_after, _, _ = detect_outliers_iqr(df_copy, col)
        n_out_after = len(outliers_after)

        summary[col] = {
            "outliers_before": n_out_before,
            "outliers_after": n_out_after,
            "winsorized": True
        }

    return df_copy, summary

clean_df, outlier_report = apply_winsorization(
    df,
    continuous_col,
    lower_pct=0.01,
    upper_pct=0.01,

```

```

    apply=True
)

print("\nOutlier Report (Before vs After Winsorization):\n")
for col, stats in outlier_report.items():
    print(f"{col:25} | before={stats['outliers_before']:5} | after={stats['outliers_

```

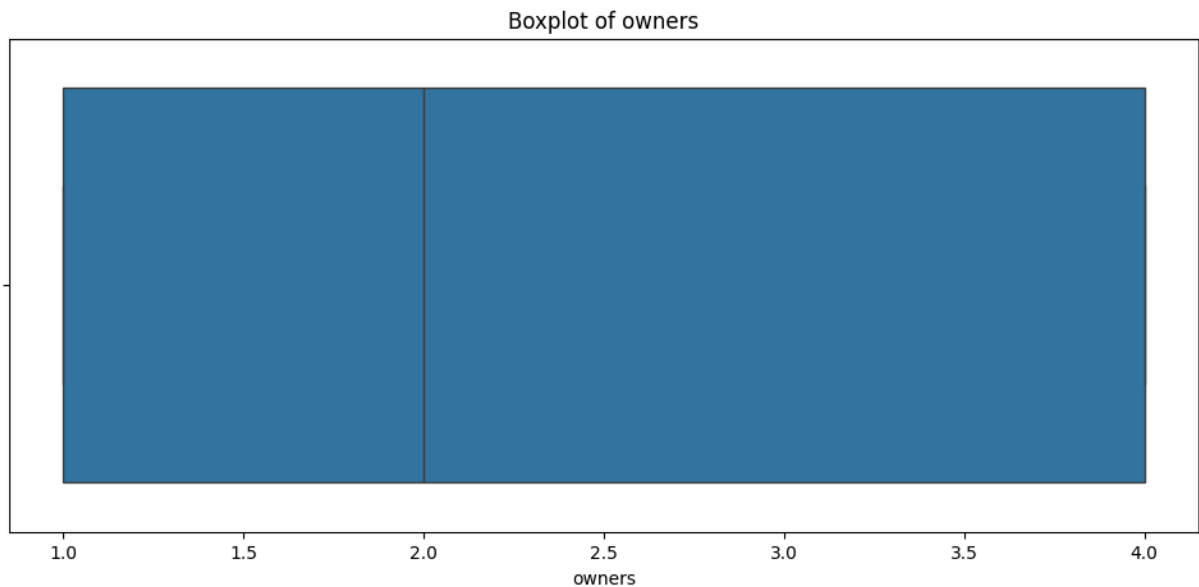
Outlier Report (Before vs After Winsorization):

owners	before= 0	after= 0	winsorized=True
year	before= 8532	after= 8532	winsorized=True
mileage	before= 6006	after= 6006	winsorized=True
power	before=20429	after=20429	winsorized=True
displacement	before=60736	after=56697	winsorized=True
super_gen_year_from	before=14722	after=14722	winsorized=True
super_gen_year_to	before= 8870	after= 8870	winsorized=True

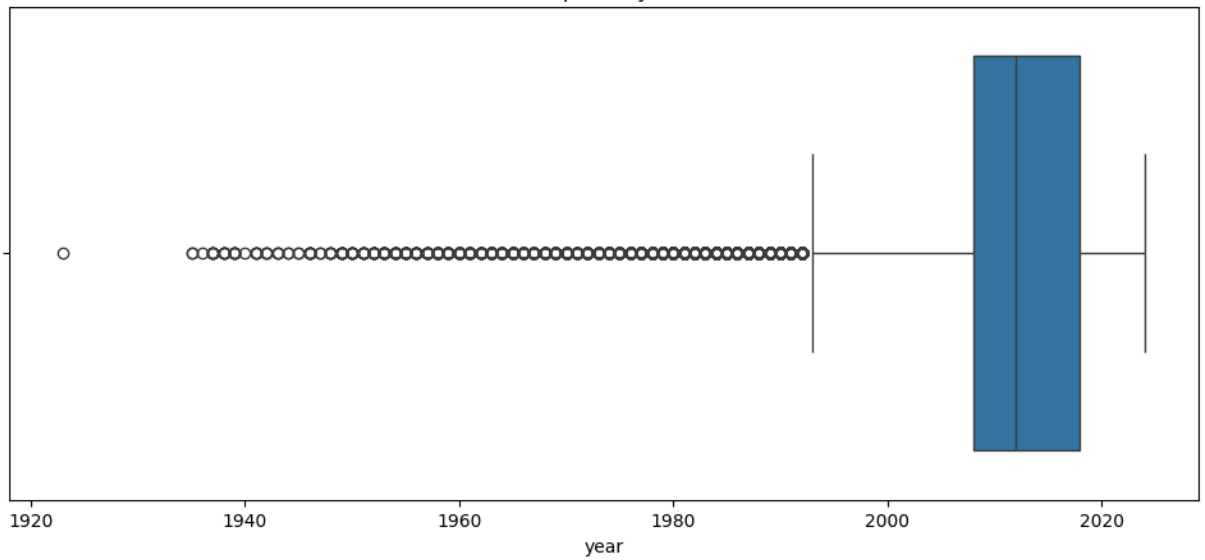
```

In [21]: for col in continuous_col:
plt.figure(figsize=(12, 5))
sns.boxplot(x=df[col])
plt.title(f"Boxplot of {col}")
plt.show()

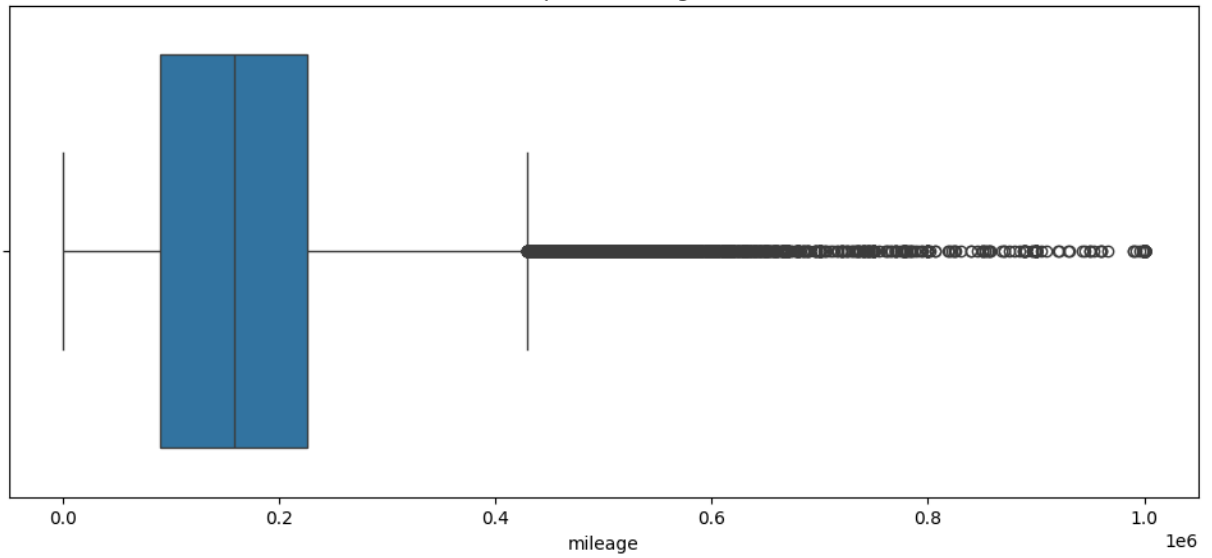
```



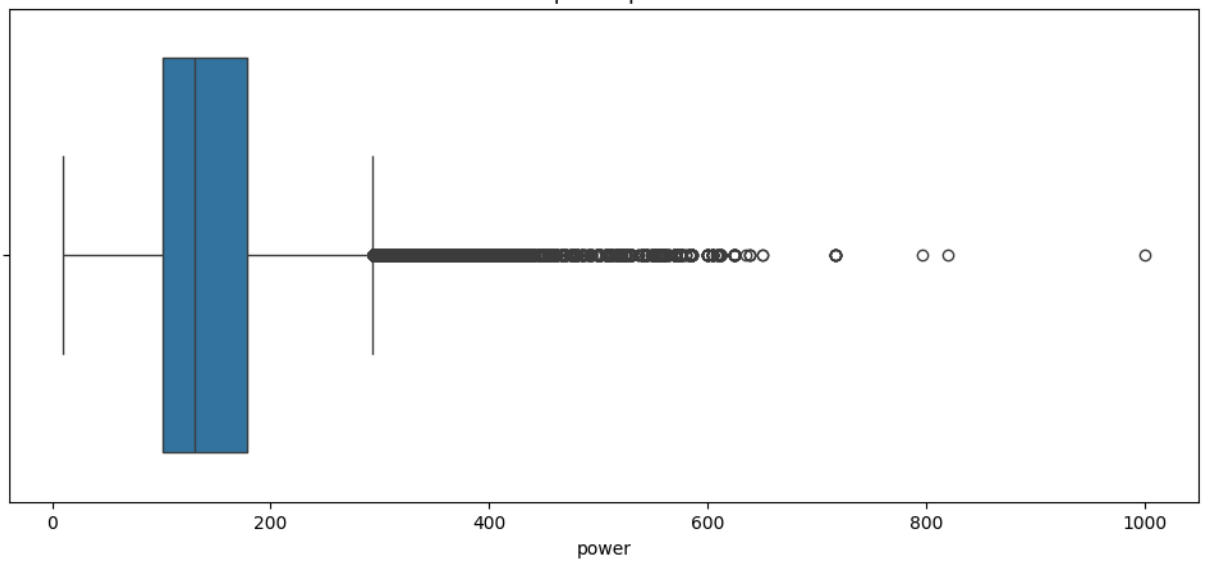
Boxplot of year



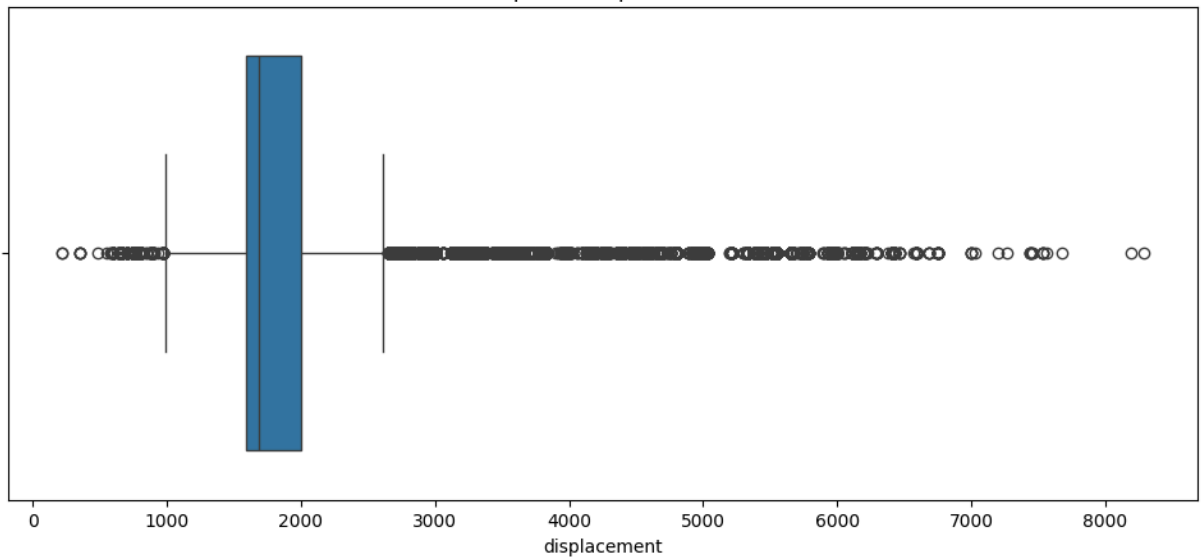
Boxplot of mileage



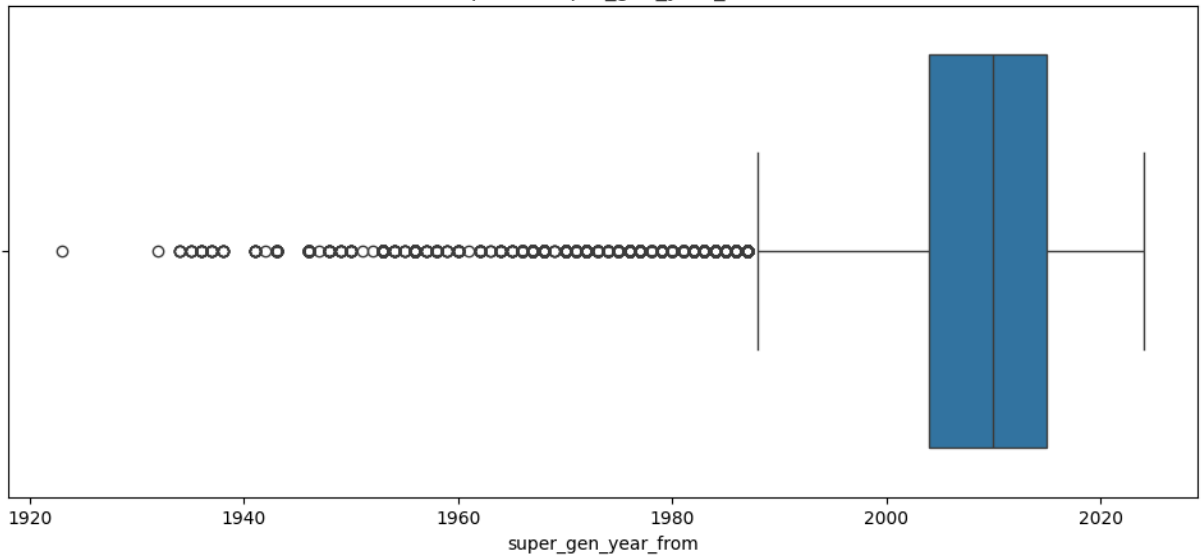
Boxplot of power



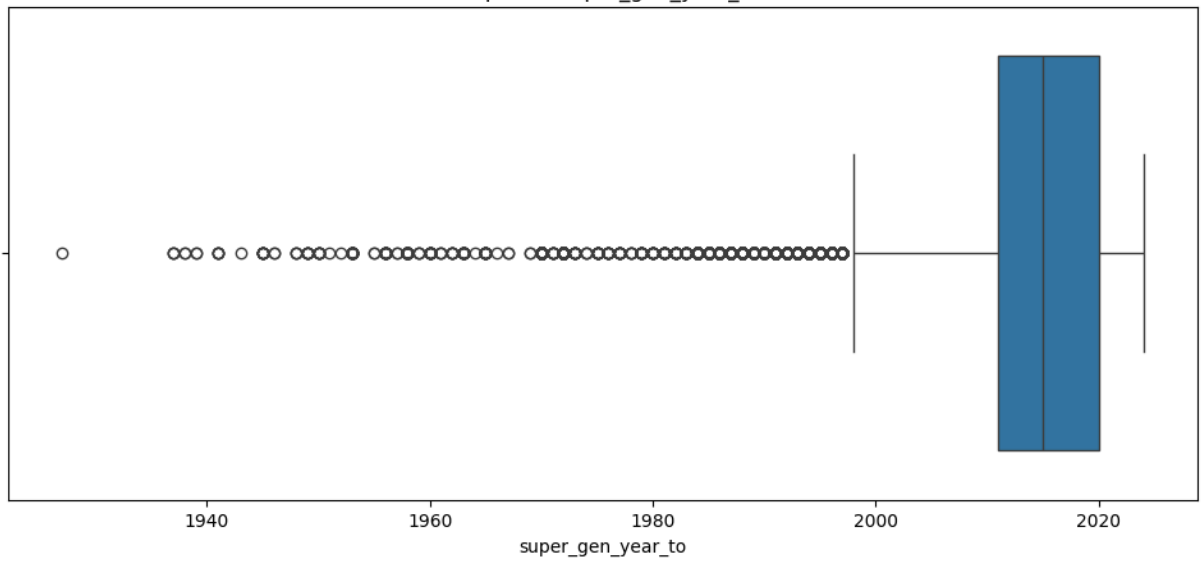
Boxplot of displacement



Boxplot of super_gen_year_from



Boxplot of super_gen_year_to



```
In [22]: def remove_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data[column] >= lower_bound) & (data[column] <= upper_bound)]

    for col in continuous_col:
        df = remove_outliers_iqr(df, col)

    print(f"Dataset shape after removing outliers: {df.shape}")
```

Dataset shape after removing outliers: (333801, 19)

```
In [23]: from scipy.stats.mstats import winsorize
import pandas as pd

def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
    return outliers, lower_bound, upper_bound

def apply_winsorization(data, continuous_cols, lower_pct=0.01, upper_pct=0.01, apply=True):
    df_copy = data.copy()
    summary = {}

    for col in continuous_cols:
        outliers, low, high = detect_outliers_iqr(df_copy, col)
        n_out_before = len(outliers)

        if not apply:
            summary[col] = {
                "outliers_before": n_out_before,
                "outliers_after": n_out_before,
                "winsorized": False
            }
            continue

        df_copy[col] = winsorize(df_copy[col], limits=[lower_pct, upper_pct])
        outliers_after, _, _ = detect_outliers_iqr(df_copy, col)
        n_out_after = len(outliers_after)

        summary[col] = {
            "outliers_before": n_out_before,
            "outliers_after": n_out_after,
            "winsorized": True
        }

    return df_copy, summary

clean_df, outlier_report = apply_winsorization(
```

```

df,
continuous_col,
lower_pct=0.01,
upper_pct=0.01,
apply=True
)

print("\nOutlier Report (Before vs After Winsorization):\n")
for col, stats in outlier_report.items():
    print(f"{col:25} | before={stats['outliers_before']:5} | after={stats['outliers_

```

Outlier Report (Before vs After Winsorization):

owners	before=	0	after=	0	winsorized=True
year	before=	0	after=	0	winsorized=True
mileage	before=	833	after=	0	winsorized=True
power	before=	11732	after=	11732	winsorized=True
displacement	before=	3	after=	0	winsorized=True
super_gen_year_from	before=	1431	after=	0	winsorized=True
super_gen_year_to	before=	0	after=	0	winsorized=True

chi squared test for categorical

```

In [24]: df_copy = df.copy()
categorical_col = df_copy.select_dtypes(include=['object', 'category']).columns

df_copy['price_bin'] = pd.qcut(df_copy['price'], q=5, labels=False)
target = 'price_bin'

chi2_results = {}
le = LabelEncoder()

for col in categorical_col:
    if col == target:
        continue
    feature_encoded = le.fit_transform(df_copy[col])
    target_encoded = le.fit_transform(df_copy[target])
    chi2_stat, p_val = chi2(feature_encoded.reshape(-1, 1), target_encoded)
    chi2_results[col] = p_val[0]

important_cols = [col for col, p in chi2_results.items() if p < 0.05]
print(important_cols)

```

```

['region', 'mark', 'model', 'steering_wheel', 'gear_type', 'engine', 'transmission',
'characteristics', 'color', 'body_type_type', 'body_type_name']

```

ANOVA TEST

```

In [25]: import pandas as pd
from scipy.stats import f_oneway

anova_results = {}

for col in categorical_col:

```

```

groups = [df['price'][df[col] == cat] for cat in df[col].dropna().unique()]
if len(groups) > 1:
    f_stat, p_val = f_oneway(*groups)
    anova_results[col] = p_val

important_cols = [col for col, p in anova_results.items() if p < 0.05]
print("Columns significantly correlated with price:", important_cols)

```

Columns significantly correlated with price: ['region', 'mark', 'model', 'steering_wheel', 'gear_type', 'engine', 'transmission', 'characteristics', 'color', 'body_type', 'body_type_name']

```

In [26]: df_encoded = df.copy()
categorical_col = df_encoded.select_dtypes(include=['object', 'category']).columns

le = LabelEncoder()
for col in categorical_col:
    df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))

```

standrization

```

In [27]: scaler = MinMaxScaler()
df_encoded[continuous_col] = scaler.fit_transform(df_encoded[continuous_col])

df_encoded

```

```

Out[27]:

```

	owners	year	price	region	mileage	mark	model	steering_wheel	gear
0	0.000000	0.903226	1460000	44	0.131423	87	817	1	
1	0.000000	0.580645	1460000	41	0.279275	110	1310	0	
2	0.333333	0.838710	1460000	20	0.171554	65	954	0	
3	0.333333	0.806452	1459000	33	0.242899	123	864	0	
4	0.000000	0.935484	1457000	4	0.078949	14	1135	0	
...	...	...	...	...	...	...	...	...	...
432220	0.333333	0.612903	290000	63	0.415394	26	811	0	
432221	1.000000	0.419355	290000	78	0.335600	66	654	0	
432222	1.000000	0.354839	287000	48	0.679574	98	771	0	
432224	0.333333	0.548387	290000	33	0.243204	132	315	0	
432225	0.666667	0.387097	290000	33	0.508931	98	728	0	

333801 rows × 19 columns



splitting train test

```
In [28]: # ==== Cell 1 - Train/Test Split + Setup ====

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# X = all features except price, y = target price
X = df_encoded.drop(columns=['price'])
y = df_encoded['price']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Dictionary to store R2 scores of each model for comparison
model_results = {}
```

train models

```
In [29]: # ==== Cell 2 - Linear Regression ====

from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()

# Simple hyperparameter space for RandomizedSearchCV
lin_param_dist = {
    'fit_intercept': [True, False],
    'positive': [True, False]
}

lin_rand_search = RandomizedSearchCV(
    estimator=lin_reg,
    param_distributions=lin_param_dist,
    n_iter=10,
    scoring='r2',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

lin_rand_search.fit(X_train, y_train)

best_lin = lin_rand_search.best_estimator_
y_pred_lin = best_lin.predict(X_test)

rmse_lin = np.sqrt(mean_squared_error(y_test, y_pred_lin))
mae_lin = mean_absolute_error(y_test, y_pred_lin)
r2_lin = r2_score(y_test, y_pred_lin)

print("=== Linear Regression ===")
print("Best params:", lin_rand_search.best_params_)
```

```

print(f"RMSE: {rmse_lin:.4f}")
print(f"MAE : {mae_lin:.4f}")
print(f"R2 : {r2_lin:.4f}")

model_results['LinearRegression'] = r2_lin

```

Fitting 3 folds for each of 4 candidates, totalling 12 fits

=== Linear Regression ===

Best params: {'positive': False, 'fit_intercept': True}

RMSE: 610481.8778

MAE : 396121.9465

R² : 0.7273

```

In [30]: # ==== Cell 3 - Ridge Regression ====

from sklearn.linear_model import Ridge

ridge = Ridge(random_state=42)

ridge_param_dist = {
    'alpha': np.logspace(-3, 3, 50)
}

ridge_rand_search = RandomizedSearchCV(
    estimator=ridge,
    param_distributions=ridge_param_dist,
    n_iter=20,
    scoring='r2',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

ridge_rand_search.fit(X_train, y_train)

best_ridge = ridge_rand_search.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print("=== Ridge Regression ===")
print("Best params:", ridge_rand_search.best_params_)
print(f"RMSE: {rmse_ridge:.4f}")
print(f"MAE : {mae_ridge:.4f}")
print(f"R2 : {r2_ridge:.4f}")

model_results['Ridge'] = r2_ridge

```

```
Fitting 3 folds for each of 20 candidates, totalling 60 fits
=== Ridge Regression ===
Best params: {'alpha': np.float64(0.21209508879201905)}
RMSE: 610481.9795
MAE : 396121.2877
R2 : 0.7273
```

```
In [31]: # ==== Cell 4 - Lasso Regression ====

from sklearn.linear_model import Lasso

lasso = Lasso(random_state=42, max_iter=10000)

lasso_param_dist = {
    'alpha': np.logspace(-4, 1, 50)
}

lasso_rand_search = RandomizedSearchCV(
    estimator=lasso,
    param_distributions=lasso_param_dist,
    n_iter=20,
    scoring='r2',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

lasso_rand_search.fit(X_train, y_train)

best_lasso = lasso_rand_search.best_estimator_
y_pred_lasso = best_lasso.predict(X_test)

rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
mae_lasso = mean_absolute_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("=== Lasso Regression ===")
print("Best params:", lasso_rand_search.best_params_)
print(f"RMSE: {rmse_lasso:.4f}")
print(f"MAE : {mae_lasso:.4f}")
print(f"R2 : {r2_lasso:.4f}")

model_results['Lasso'] = r2_lasso
```

```
Fitting 3 folds for each of 20 candidates, totalling 60 fits
=== Lasso Regression ===
Best params: {'alpha': np.float64(4.941713361323838)}
RMSE: 610482.3137
MAE : 396117.6861
R2 : 0.7273
```

```
In [32]: # ==== Cell 5 - Elastic Net Regression ====

from sklearn.linear_model import ElasticNet
```

```

elastic = ElasticNet(random_state=42, max_iter=10000)

elastic_param_dist = {
    'alpha': np.logspace(-4, 1, 50),
    'l1_ratio': np.linspace(0.1, 0.9, 9)
}

elastic_rand_search = RandomizedSearchCV(
    estimator=elastic,
    param_distributions=elastic_param_dist,
    n_iter=30,
    scoring='r2',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

elastic_rand_search.fit(X_train, y_train)

best_elastic = elastic_rand_search.best_estimator_
y_pred_elastic = best_elastic.predict(X_test)

rmse_elastic = np.sqrt(mean_squared_error(y_test, y_pred_elastic))
mae_elastic = mean_absolute_error(y_test, y_pred_elastic)
r2_elastic = r2_score(y_test, y_pred_elastic)

print("=== Elastic Net ===")
print("Best params:", elastic_rand_search.best_params_)
print(f"RMSE: {rmse_elastic:.4f}")
print(f"MAE : {mae_elastic:.4f}")
print(f"R²   : {r2_elastic:.4f}")

model_results['ElasticNet'] = r2_elastic

```

Fitting 3 folds for each of 30 candidates, totalling 90 fits

=== Elastic Net ===

Best params: {'l1_ratio': np.float64(0.8), 'alpha': np.float64(0.0005179474679231213)}

RMSE: 610506.3888

MAE : 396044.9064

R² : 0.7273

In [33]: # ==== Cell 6 - Decision Tree Regressor ====

```

from sklearn.tree import DecisionTreeRegressor

dt = DecisionTreeRegressor(random_state=42)

dt_param_dist = {
    'max_depth': [None] + list(range(3, 21)),
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 8],
    'max_features': ['auto', 'sqrt', 'log2', None]
}

```



```

dt_rand_search = RandomizedSearchCV(
    estimator=dt,
    param_distributions=dt_param_dist,
    n_iter=40,
    scoring='r2',
    cv=3,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

dt_rand_search.fit(X_train, y_train)

best_dt = dt_rand_search.best_estimator_
y_pred_dt = best_dt.predict(X_test)

rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
mae_dt = mean_absolute_error(y_test, y_pred_dt)
r2_dt = r2_score(y_test, y_pred_dt)

print("=== Decision Tree Regressor ===")
print("Best params:", dt_rand_search.best_params_)
print(f"RMSE: {rmse_dt:.4f}")
print(f"MAE : {mae_dt:.4f}")
print(f"R²  : {r2_dt:.4f}")

model_results['DecisionTree'] = r2_dt

```

Fitting 3 folds for each of 40 candidates, totalling 120 fits

=== Decision Tree Regressor ===

Best params: {'min_samples_split': 15, 'min_samples_leaf': 2, 'max_features': None, 'max_depth': 20}

RMSE: 291939.2499

MAE : 173310.0359

R² : 0.9376

In [34]: *# ===== Cell 8 - Simple Gradient Boosting Regressor =====*

```

from sklearn.ensemble import GradientBoostingRegressor

# Simple, single configuration (no RandomizedSearch, no CV)
gbr = GradientBoostingRegressor(
    random_state=42,
    n_estimators=200,
    learning_rate=0.1,
    max_depth=3,
    subsample=0.8
)

gbr.fit(X_train, y_train)

y_pred_gbr = gbr.predict(X_test)

rmse_gbr = np.sqrt(mean_squared_error(y_test, y_pred_gbr))
mae_gbr = mean_absolute_error(y_test, y_pred_gbr)
r2_gbr = r2_score(y_test, y_pred_gbr)

```

```

print("=== Gradient Boosting Regressor (simple) ===")
print("Params used:")
print(gbr.get_params())
print(f"RMSE: {rmse_gbr:.4f}")
print(f"MAE : {mae_gbr:.4f}")
print(f"R2 : {r2_gbr:.4f}")

model_results['GradientBoosting'] = r2_gbr
best_gbr = gbr

```

=== Gradient Boosting Regressor (simple) ===

Params used:

```

{'alpha': 0.9, 'ccp_alpha': 0.0, 'criterion': 'friedman_mse', 'init': None, 'learning_rate': 0.1, 'loss': 'squared_error', 'max_depth': 3, 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'n_estimators': 200, 'n_iter_no_change': None, 'random_state': 42, 'subsample': 0.8, 'tol': 0.0001, 'validation_fraction': 0.1, 'verbose': 0, 'warm_start': False}
RMSE: 333263.6231
MAE : 207802.0421
R2 : 0.9187

```

In [35]: # ===== Cell 12 - Plot Comparison of R² Scores =====

```

import matplotlib.pyplot as plt

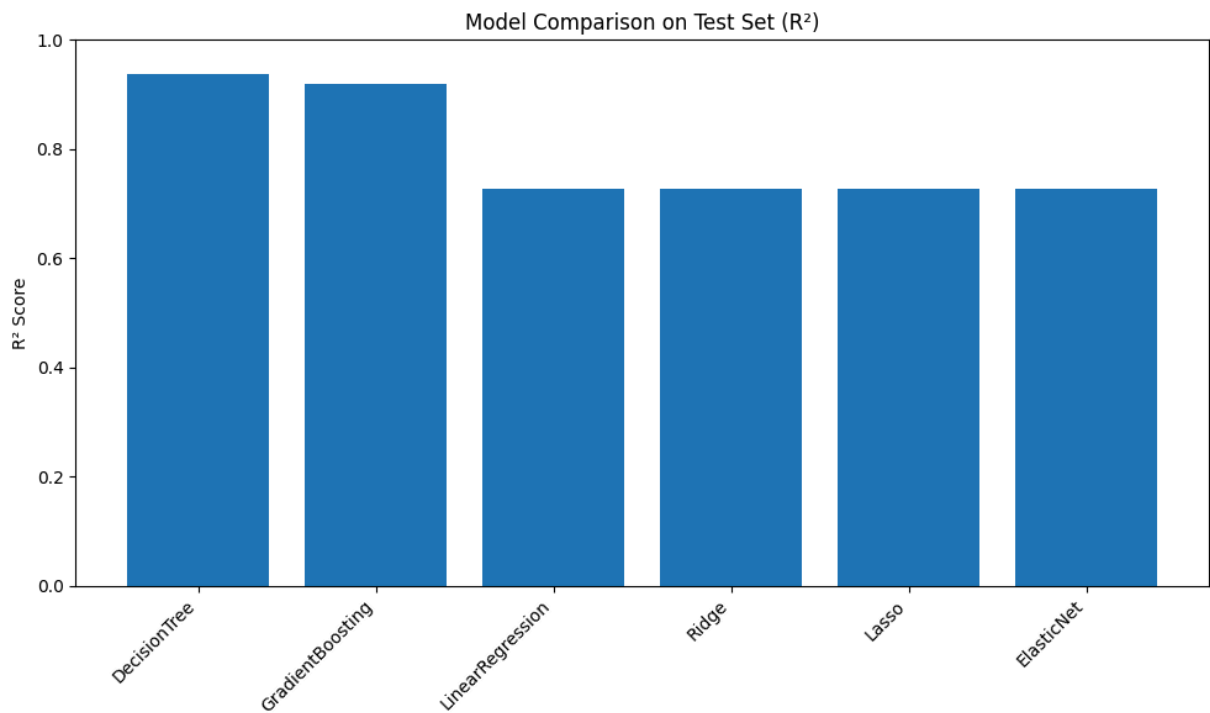
# Sort models by R2 descending
sorted_results = dict(sorted(model_results.items(), key=lambda x: x[1], reverse=True))

names = list(sorted_results.keys())
scores = list(sorted_results.values())

plt.figure(figsize=(10, 6))
plt.bar(range(len(names)), scores)
plt.xticks(range(len(names)), names, rotation=45, ha='right')
plt.ylabel('R2 Score')
plt.title('Model Comparison on Test Set (R2)')
plt.ylim(0, 1) # Adjust if needed depending on your results
plt.tight_layout()
plt.show()

print("R2 scores by model:")
for name, score in sorted_results.items():
    print(f"{name:20s}: {score:.4f}")

```



R^2 scores by model:

DecisionTree	: 0.9376
GradientBoosting	: 0.9187
LinearRegression	: 0.7273
Ridge	: 0.7273
Lasso	: 0.7273
ElasticNet	: 0.7273

saving models and encoder

```
In [36]: # ==== Cell A - Save Encoders, Scaler, and Column Order ====

import joblib
from sklearn.preprocessing import LabelEncoder

# 1) Re-fit one LabelEncoder per categorical column on the ORIGINAL df
label_encoders = {}
categorical_cols = df.select_dtypes(include=["object", "category"]).columns

for col in categorical_cols:
    le = LabelEncoder()
    le.fit(df[col].astype(str))    # same mapping used when you encoded df_encoded
    label_encoders[col] = le

# Save all label encoders in one file
joblib.dump(label_encoders, "label_encoders.pkl")
print("Saved label_encoders.pkl")

# 2) Save the MinMaxScaler you already fitted on continuous_col
joblib.dump(scaler, "minmax_scaler.pkl")
print("Saved minmax_scaler.pkl")

# 3) Save the feature column order used for training (X columns)
```

```
joblib.dump(list(X.columns), "columns.pkl")
print("Saved columns.pkl")
```

Saved label_encoders.pkl
Saved minmax_scaler.pkl
Saved columns.pkl

In [37]: *# ===== Cell B - Save Trained Models and Best Model =====*

```
import joblib

# Dictionary of all trained regression models
trained_models = {
    "LinearRegression": best_lin,
    "Ridge": best_ridge,
    "Lasso": best_lasso,
    "ElasticNet": best_elastic,
    "DecisionTree": best_dt,
    "GradientBoosting": best_gbr
}

# 1) Save each model separately
for name, model in trained_models.items():
    filename = f"{name}.pkl"
    joblib.dump(model, filename)
    print(f"Saved {filename}")

# 2) Also save the best model (based on R2 in model_results)
best_model_name = max(model_results, key=model_results.get)
best_model = trained_models[best_model_name]

joblib.dump(best_model, "best_model.pkl")
print(f"Best model is: {best_model_name}")
print("Saved best_model.pkl")
```

Saved LinearRegression.pkl
Saved Ridge.pkl
Saved Lasso.pkl
Saved ElasticNet.pkl
Saved DecisionTree.pkl
Saved GradientBoosting.pkl
Best model is: DecisionTree
Saved best_model.pkl